

Node Js

- Node.js is a server-side runtime environment built on Google Chrome's JavaScript Engine (V8 Engine).
- Node.js was developed by Ryan Dahl in 2009 and its latest version is v20.9.0.
- Node.js is a cross-platform (run on Windows, Linux, Unix, macOS, and more), open-source, back-end JavaScript runtime environment, that executes JavaScript code outside a web browser.

Features of Node.js

- Following are some of the important features that make Node.js the first choice of software architects.
- **Asynchronous and Event Driven** – All APIs of Node.js library are asynchronous, that is, non-blocking. It essentially means a Node.js based server never waits for an API to return data. The server moves to the next API after calling it and a notification mechanism of Events of Node.js helps the server to get a response from the previous API call.
- **Very Fast** – Being built on Google Chrome's V8 JavaScript Engine, Node.js library is very fast in code execution.

Features of Node.js

- **Single Threaded but Highly Scalable** – Node.js uses a single threaded model with event looping. Event mechanism helps the server to respond in a non-blocking way and makes the server highly scalable as opposed to traditional servers which create limited threads to handle requests. Node.js uses a single threaded program and the same program can provide service to a much larger number of requests than traditional servers like Apache HTTP Server.
- **No Buffering** – Node.js applications never buffer any data. These applications simply output the data in chunks.

Local Environment Setup

- Node.js can be installed on different OS platforms such as Windows, Linux, Mac OS X, etc
- Following are the tools .
 - The Node.js binary installer.
 - Node Package Manager (NPM).
 - IDE or Text Editor.
- Binaries for various OS platforms are available on the downloads page of the official website of Node.js.

<https://nodejs.org/en/download>

Node.js - REPL Terminal

- The Node.js runtime has a built-in interactive shell, in which you can execute instructions one at a time.
- The Node.js interactive REPL terminal is like the Powershell or Command prompt terminal, or a bash terminal in Linux.
- The Node.js interactive shell works on the principle of **REPL**, which is an acronym for READ, EVALUATE, PRINT and LOOP.
- **Read** – Reads user's input, parses the input into JavaScript data-structure, and stores in memory.
- **Eval** – Takes and evaluates the data structure.
- **Print** – Prints the result.
- **Loop** – The terminal is ready to receive next input from the user.

Node.js - REPL Terminal

- **Underscore Variable** : You can use underscore (`_`) to get the last result.

Node.js - REPL Terminal

Sr.No	Dot Commands & Description
1	.help shows the dot commands help
2	.editor enables editor mode, to write multiline JavaScript code with ease. Once you are in this mode, enter ctrl-D to run the code you wrote.
3	.break when inputting a multi-line expression, entering the .break command will abort further input. Same as pressing ctrl-C.
4	.clear resets the REPL context to an empty object and clears any multi-line expression currently being input.
5	.load loads a JavaScript file, relative to the current working directory
6	.save saves all you entered in the REPL session to a file (specify the filename)
7	.exit exits the repl (same as pressing ctrl-C two times)

Node.js - Command Line Options

- `:\nodejs> node`
- `:\nodejs> node -i`
- `:\nodejs> node --interactive`
- `:\nodejs> node demo.js`
- `:\nodejs> node --help`
- `:\nodejs> node -v`
- `:\nodejs> node --version`
- `:\nodejs> node -e "console.log(1+2)"`
- `:\nodejs> node --eval "console.log(1+2)"`

Node.js - NPM

- **NPM** – an acronym for Node Package Manager, refers to the Command line utility to install Node.js packages, perform version management and dependency management of Node.js packages.
- **npm -v** : This command will display version of npm.
- **Install Package Locally :**

```
npm install <Module Name>
```

Use **--save** at the end of the install command to add dependency entry into package.json of your application.

Node.js - NPM

- **Install Package Globally** : Globally installed packages/dependencies are stored in system directory. Such dependencies can be used in CLI (Command Line Interface) function of any node.js but cannot be imported using `require()` in Node application directly.

```
npm install <Module Name> -g
```

- The module will be installed globally, for Windows, the path is
C:\Users\your-username\AppData\Roaming\npm\node_modules.

Node.js - NPM

➤ Update Package :

```
npm update <package name>
```

➤ Uninstall Packages :

```
npm uninstall <package name>
```

Node.js -Module

- A module in Node.js is a collection of independent and reusable code that can be imported into any Node.js application.
- A Node.js module is a library of functions, classes and other reusable code, stored in one or more .js files.
- Modules enable a modular and structured approach for developing a Node.js application.
- Instead of putting all the functions, classes and methods of an application in a single .js file, these resources are arranged in separate files (called modules) based on their relevance.
- All the server-side processes are handled by the relevant Node.js module imported into the application by the `require()` function.

Node.js -Module

- In Node, the `module.exports` is utilized to expose literals, functions, or objects as modules. This mechanism enables the inclusion of JavaScript files within Node.js applications. The `module` serves as a reference to the current module, and `exports` is an object that is made accessible as the module's public interface.
- **Syntax:**
 - `module.exports = literal | function | object`
- **Import Modules:** To include a module, use the **require()** function with the name of the module:
- **Syntax:**
 - `var temp = require('module_name');`

Node.js HTTP Module

- Node.js has a built-in module called HTTP, which allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP).
- An HTTP server is a computer program or a software component of another program that acts as a server in a client-server model by implementing the server portion of the HTTP and/or HTTPS network protocols.
- An HTTP server waits for incoming client requests and responds to each one by returning desired information, including the delivery of the requested web page, or by returning an HTTP error message.

Node.js HTTP Module

➤ **Creating a Basic HTTP Server :**

- First load the http module which is a core Node.js module.

```
const http = require("http");
```

- Create two constants two stores of the value of the hostname and the port:

```
const host = 'localhost';  
const port = 8000;
```

- The localhost is a private address used by computers to refer to themselves. It is equivalent to the internal IP address 127.0.0.1 and is only accessible to the local computer and not to any local networks

Node.js HTTP Module

- The port is a number that servers use to connect to a specific process on our IP address as an endpoint, our web server will run on port 8000.
- The server is bound to the host and port, and hence you will be able to access it on `http://localhost:8000`.

Node.js HTTP Module

- Create a function **requestListener** to handle the incoming requests.
- This function requires two parameters: a request object and a response object.
- The request object stores all of the data from the incoming HTTP request.
- The response object is responsible for returning HTTP responses to the server, and it contains all the details about the response.

```
const requestListener = function (req, res) {  
  res.writeHead(200)  
  res.end('Welcome user!!')  
}
```

- The function will return Welcome user!! as a response to the client.

Node.js HTTP Module

- **res.writeHead(200)**, specifies the HTTP status code of the response.
- HTTP status codes represent how well a server handled an HTTP request.
- The status code 200 corresponds to OK in this instance.
- **res.end("Welcome user!!")**, returns the HTTP response to the client who requested it. This function returns any data to be sent from the server. It is returning text data in this case.

Node.js HTTP Module

- First, the server object is created using the **createServer()** function.
- The server object accepts all the requests and passes them to the **requestListener()** function.
- The server is then attached to the port and host using the **server.listen()** function.
- The listen() function takes the port, host and the callback function as arguments. The callback function is called when the server begins to listen.

```
const server = http.createServer(requestListener)
server.listen(port, host, () => {
  console.log(`Server is running on http://${host}:${port}`)
})
```

Node.js HTTP Module



```
const http = require('http')

const host = 'localhost'
const port = 8000

const requestListener = function (req, res) {
  res.writeHead(200)
  res.end('Welcome user!!')
}

const server = http.createServer(requestListener)
server.listen(port, host, () => {
  console.log(`Server is running on http://${host}:${port}`)
})
```